

Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

The problem, in one breath

- Bayesian inference usually means re-running a sampler *from scratch* for every new dataset.
- **Neural Bayes estimators** avoid that: train once, then predict in a single forward pass.
- Training them is *itself* a Monte Carlo computation — so can classic **variance reduction** make training cheaper?

⇒ One technique comes with a **guarantee**; a second is more general but does not.

Bayesian inference is expensive — and repeated

- For each dataset $\mathbf{y}$ we want the posterior $p(\boldsymbol{\theta} | \mathbf{y})$.
- The standard tool — MCMC — is accurate but must be **re-run from scratch** for every new $\mathbf{y}$.

$$\mathbf{y}_1 \rightarrow \boxed{\text{MCMC}} \rightarrow p(\boldsymbol{\theta} | \mathbf{y}_1) \quad \mathbf{y}_2 \rightarrow \boxed{\text{MCMC}} \rightarrow p(\boldsymbol{\theta} | \mathbf{y}_2) \quad \dots$$

the same expensive computation, paid again every time

Neural Bayes estimators: pay once

- Under squared-error loss the best estimator is the **posterior mean** — a fixed function of the data.
- Approximate that function *once* with a neural network $\hat{\theta}_{\gamma}(\mathbf{y})$, trained on simulated pairs $(\theta, \mathbf{y})$.
- Deployment for any new dataset is a **single forward pass**.



trained once on simulated $(\theta, \mathbf{y})$ pairs

The observation this thesis starts from

Training minimises the **Bayes risk**

$$L(\gamma) = \mathbb{E}_{\theta, \mathbf{y}}[\|\theta - \hat{\theta}_{\gamma}(\mathbf{y})\|^2].$$

- In practice this expectation is a **Monte Carlo average** over simulated draws.
- So the loss — and the gradient that drives training — is a **random quantity with variance**.
- That variance comes from *how we sample the loss*, not from the inference problem. *So we can reduce it.*

A trick for killing Monte Carlo noise

- You are estimating an average by sampling. Suppose part of it you can compute **exactly**.
- Then replace that sampled piece by its **exact conditional mean** instead of drawing it.
- Averaging out a random draw can only *remove* variance — never add it.

noisy draws → replace one coordinate by its exact mean → tighter estimate

Statisticians call this **Rao–Blackwellisation**.

The Rao–Blackwellised loss

Split the parameters $\theta = (\tau, \beta)$. Suppose β has a **tractable conditional posterior** given $(\tau, \mathbf{y})$ (here: conjugate).

sampld target $\beta \longrightarrow \mathbb{E}[\beta | \tau, \mathbf{y}]$ (its conditional posterior mean)

- We integrate β out of the loss analytically instead of sampling it.
- The leftover conditional variance is a **constant** — it drops out of the gradient.

The guarantee

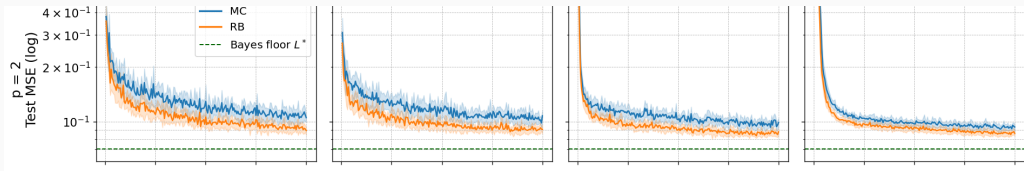
The Rao–Blackwellised gradient is a **conditional average** of the standard one. The law of total variance then gives

$$\text{Var}(\nabla_{\text{RB}}) \preceq \text{Var}(\nabla_{\text{MC}}) \quad (\text{positive semi-definite gap}).$$

- It can **never** increase the gradient variance — for *any* model, *any* architecture.
- For a linear network the same reduction carries through to the **fitted weights** $\hat{\gamma}$ themselves.

For free, provably: never worse, usually better.

Does it pay off? Bayesian linear regression



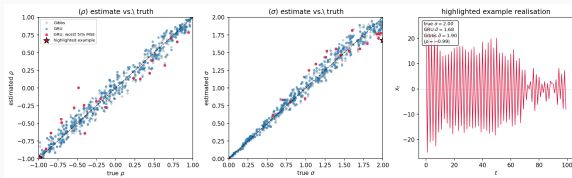
MC (blue) vs RB (orange) vs Bayes floor L^* (green) — across batch sizes 4, 16, 64, 256.

lower loss in
every grid cell

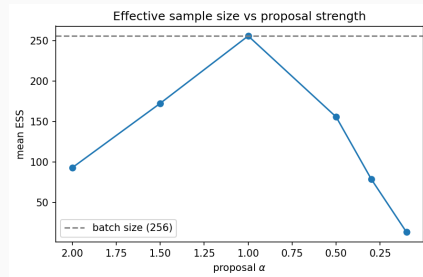
closes $\sim$ **half** (52%)
of the gap to L^*

matches the standard NBE
at $\frac{1}{4}$ **the batch size**

A contrasting lever: importance sampling



The GRU already tracks the Gibbs (Bayes) reference across the prior — **little error left to recover**.



Push the proposal hard and the effective sample size **collapses** — only adding noise.

What separates the two techniques

	Rao–Blackwell	Importance sampling
needs	a conjugate block	nothing special
variance guarantee	yes, always	none
carries to fitted weights	yes (linear net)	no

Where conjugacy exists, Rao–Blackwell is the safer, stronger lever.

Takeaway

- Training a neural Bayes estimator is Monte Carlo — so the Monte Carlo variance-reduction toolbox applies to *training*.
- **Rao–Blackwellisation** is a cheap, reliable lever: a *provable* cut in gradient variance, and real gains toward the Bayes floor.

Conjugacy — the classical trick behind Gibbs sampling — is just as useful for training the estimators meant to replace it.

Thank you

Questions?

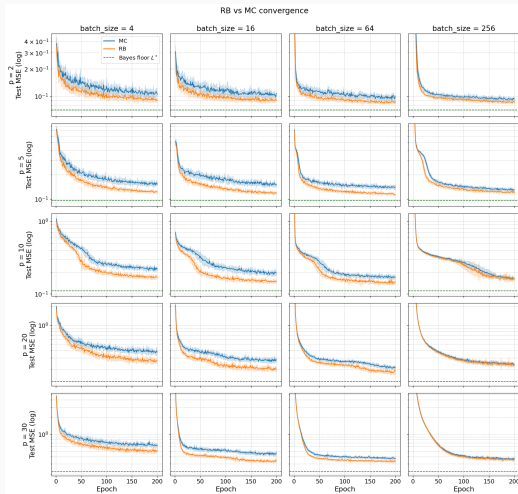
Backup: why the variance can only shrink

Write the standard gradient g and condition on $(\tau, \mathbf{y})$. By the law of total variance,

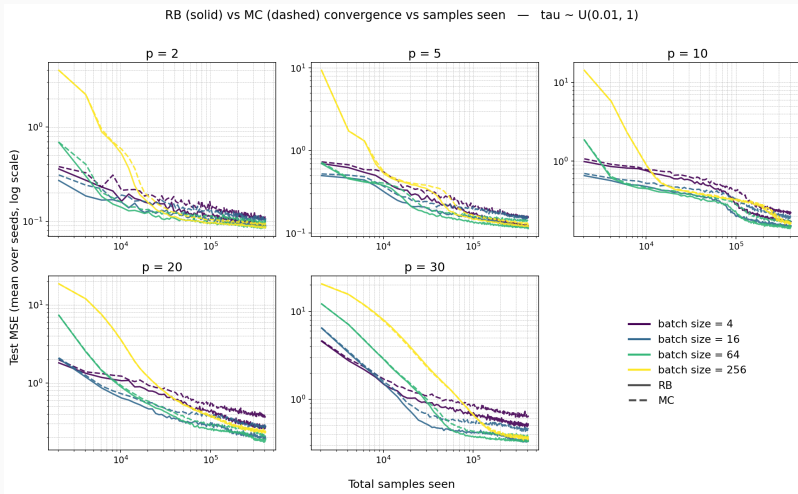
$$\text{Var}(g) = \underbrace{\text{Var}(\mathbb{E}[g | \tau, \mathbf{y}])}_{\text{Var}(\nabla_{\text{RB}})} + \underbrace{\mathbb{E}[\text{Var}(g | \tau, \mathbf{y})]}_{\succeq 0}.$$

The second term is positive semi-definite, so $\text{Var}(\nabla_{\text{RB}}) \preceq \text{Var}(g)$.

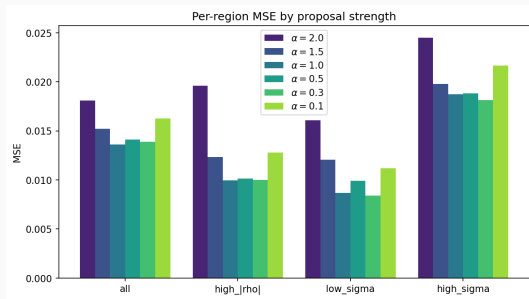
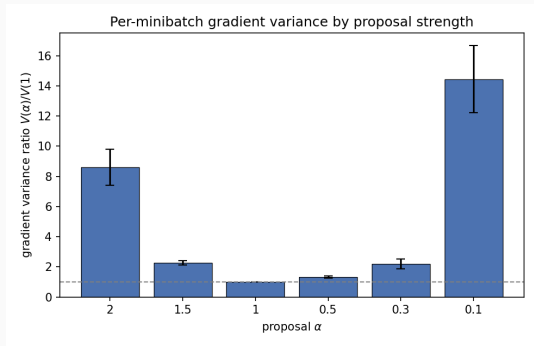
Backup: RB vs MC across the whole grid



Backup: convergence vs total samples seen



Backup: importance-sampling diagnostics



Left: per-minibatch gradient variance blows up as the proposal is pushed away from uniform. Right: the error lives at the boundary, which the proposals do not target.

Backup: models & architectures

- **RB study:** Bayesian linear regression, $\theta = (\tau, \beta)$; β (coefficients) integrated out, $\tau \sim \text{U}(0.01, 1)$ sampled; multilayer perceptron; Gibbs sampler for the Bayes floor L^* .
- **IS study:** AR(1) process, $T = 100$; parameters (ρ, σ) ; recurrent (GRU) network; Beta(α, α) proposal family on ρ .